Management Control for Financial Institutions
Assignment D

Pim van den Bos s2619652

Which damping mechanisms does the solidarity-based contribution scheme (SPR) under the Wtp contain to reduce volatility in pension outcomes, and how large are their individual and combined contributions to this reduction?

UNIVERSITY
OF TWENTE.

Department of IEBIS
University of Twente

3th of July, 2026

1 Introduction

The Dutch pension system is the last years being reformed through the *"Wet toekomst pensioenen"* (Wtp). In the old system many occupational pension schemes were DB (defined benefit), which means that participants expected a relatively stable pension benefit. Low interest rates, ageing and financial market shocks, however, made it increasingly difficult for pension funds to maintain these promises and index pensions consistently (Bovenberg & Nijman, 2009; Ministerie van Sociale Zaken en Werkgelegenheid, 2022). Therefore the Wtp introduces a more DC (defined contribution) oriented system in which pension outcomes depend more directly on contributions, investment returns and interest-rate developments (De Nederlandsche Bank, 2026b).

Within the *"solidaire premieregeling"* (SPR) risks are still shared collectively, but in a different way. Pension assets are invested collectively and returns are allocated to participants through pre-defined rules. Age dependent allocation of excess returns and protective returns for interest-rate risk for example (De Nederlandsche Bank, 2023b). This makes the system more transparent, but also means that participants could be more directly exposed to financial market volatility than within the old pension system.

2 Problem Formulation

The Wtp changes how risks are managed in Dutch occupational pensions. Under the old DB oriented system, pension funds used collective buffers, funding-ratio rules and conditional indexation to absorb shocks. This also made the distribution of risks between participants less transparent, especially when low funding ratios led to limited indexation or benefit cuts (Bovenberg & Nijman, 2009). Under the Wtp pension outcomes become more directly linked to contributions, investment returns and interest-rate developments (De Nederlandsche Bank, 2026b).

This creates a challenge for the *"solidaire premieregeling"* (SPR). The SPR maintains collective investment and risk sharing, but the returns are allocated to the participants through predefined rules, including protective returns and excess returns (De Nederlandsche Bank, 2023b; Werker, 2022). Now participants may be (more) directly exposed to market volatility, especially when investment returns or interest rates change. This is especially relevant for older participants and retired people, because volatility can affect their expected or actual pension income directly.

The SPR uses damping mechanisms such as protective returns, shock spreading and the solidarity reserve to reduce this volatility (De Nederlandsche Bank, 2023b; Werker, 2022). These mechanisms stabilize pension outcomes, but they do not completely eliminate the risks. They influence how risks and returns are allocated among groups of participants. For example, using reserves or spreading shocks could improve short-term stability, but can also create redistribution between generations (van Bilsen et al., 2022).

The problem is that damping mechanisms must balance two objectives at once: reducing volatility in pension outcomes, and keeping risk sharing transparent, fair, and sustainable. This paper focuses on how the damping mechanisms in the SPR reduce volatility, and what trade-offs they create for risk management and intergenerational risk sharing.

3 Gaps in the Literature

The literature discusses several parts of the new Dutch pension contract, but often fragmented and not in a complete way. Technical sources explain how protective returns, excess returns and project returns work within the SPR (Werker, 2022). Other studies focus on the redistributive effects of specific design choices and especially the role of the solidarity reserve and the possibility of intergenerational transfers (van Bilsen et al., 2022).

The problem is that less attention is given to the combined role of these mechanisms as tools to dampen volatility in pension outcomes. Most sources analyse individual instruments, while the interaction between protective returns, shock spreading and the solidarity reserve remain less explicit. This matters because these mechanisms could stabilise pensions for some participants, while they shift risks or returns to other groups.

A second gap is that the management control perspective is relatively limited. The design of damping mechanisms is not only a technical issue, but also a governance issue. Pension funds must decide how much volatility is acceptable, how risks are allocated between generations and how these choices can be explained to participants.

Research Question

The research question guiding our report is:
Which damping mechanisms does the solidarity-based contribution scheme (SPR) under the Wtp contain to reduce volatility in pension outcomes, and how large are their individual and combined contributions to this reduction?

4 Background Information

In this chapter, the two contract variants within the new defined-contribution pension arrangement in The Netherlands are discussed: the SPR and the FPR . Furthermore, the six mechanisms built in the SPR variant are discussed, which allow collective risk-sharing and volatility minimization.

4.1 SPR and FPR

The new Dutch pension framework introduced by the Wtp, which entered into force on 1 July 2023, allows pension funds to choose between two contract variants when transitioning from defined-benefit to defined-contribution arrangements: the solidaire premieregeling (SPR, solidarity-based contribution scheme) and the flexibele premieregeling (FPR, flexible contribution scheme). Both replace the traditional guaranteed benefit promise with an individual approach. However, both arrangements differ in terms of how investment risk is shared and management across the collective.

In the SPR, all accrued capital is pooled in collective investment portfolios managed by the pension fund. Participants do not hold individual investment accounts. Instead, returns are allocated to individual pension capitals according to a predetermined, age-dependent distribution rule (levenscyclusbeleid, or life-cycle policy), while a collective solidarity reserve absorbs part of the investment risk and smooths outcomes across generations (De Nederlandsche Bank, 2023d). The FPR, on the other hand, gives each participant an individual investment account in which they can choose their own investment mix and risk profile from a range of options provided by the pension fund. Biometric and longevity risk are hedged through a collective risk-sharing arrangement (collectieve risicodeling) rather than a solidarity reserve (Werken aan ons Pensioen, n.d.-a).

This report focuses on the SPR, since it is the variant adopted by the large majority of Dutch pension funds undergoing the transition and is the standard against which the new framework's risk-sharing and damping mechanisms are typically assessed (Balter & Werker, 2021; De Nederlandsche Bank, 2023d).

4.2 Damping Mechanisms

The SPR contains six mechanisms designed to dampen the volatility of investments and interest rate risk on individual pension capitals and benefits. These six mechanism are discussed below.

1. Spreading Mechanism (Spreidingsmechanisme)

The Spreading Mechanism ensures the possibility to spread financial and biometric risks over a maximum period of 10 years. This allows investment risks to be spread out over several years, instead of being reflected in pension benefits immediately. This ensures volatility in pension outcomes is reduced, especially for retirees who are already receiving pension payments (De Nederlandsche Bank, 2022).

2. Solidarity Reserve (Solidariteitsreserve)

The Solidarity Reserve is a collective buffer that belongs to all participants in a pension fund. It is used to absorb financial shocks and smooth pension outcomes over time, thereby reducing the

impact of market downturns on pension benefits. By building up reserves in favorable years and using them in adverse years, the system facilitates intergenerational risk sharing and limits abrupt changes in pension payments. The reserve is funded by a deduction from contributions and/or investment returns. It is bounded between 0% and 15% of total pension assets (De Nederlandsche Bank, 2023c; van Bilsen & Mehlkopf, 2026).

3. Protection Return (Beschermingsrendement)

The Protection Return protects participants against interest-rate risk by dividing total investment returns into a protection return and an excess return (overrendement). The protection return compensates for changes in interest rates, especially for participants close to retirement and retirees, whose pensions are more sensitive to interest-rate movements. This mechanism therefore increases stability for participants approaching or in retirement (Werker, 2022). Protection against interest-rate risk can be implemented either via a separate protection portfolio (direct method) or directly within the collective investment portfolio (indirect method) (De Nederlandsche Bank, 2023d).

4. Age-Dependent Allocation of Excess Return (Leeftijdsafhankelijke Toedeling van het Overrendement)

The Age-Dependent Allocation of Excess Returns distributes investment returns across participants based on age. Younger participants receive a larger share of excess returns and therefore bear more investment risk, as they have a longer horizon to recover from potential losses. Older participants receive a smaller share of excess returns, reducing their exposure to volatility and increasing stability near retirement. If pension payouts decline due to negative excess returns, adjustments can be made via the solidarity reserve (De Nederlandsche Bank, 2023d).

5. Projection Return (Projectierendement)

The Projection Return is used to determine the conversion of pension capital into pension benefits from retirement, subject to a maximum under the Pension Act (Wet toekomst pensioenen, 2022, Art. 63a). It is not a realized return but a planning assumption used to derive projected pension outcomes and communicate expected income. The projection return is generally fixed at retirement but may be adjusted under specific conditions, provided it is not used to soften the effect of adverse financial shocks (De Nederlandsche Bank, 2023a; Wet toekomst pensioenen, 2022, Art. 17a, para. 3).

6. Collective Payout Phase (Collectieve Uitkeringsfase)

Finally, the Collective Payout Phase allows retirees to continue sharing investment and longevity risks collectively after retirement. Instead of bearing these risks individually, they are pooled within the pension fund, which smooths pension payments over time. This collective structure improves risk diversification and results in more stable pension income and lower exposure to longevity risk compared with individual arrangements (Werken aan ons Pensioen, n.d.-b).

5 Methodology

In this chapter, the methodology used to illustrate the effects of the damping mechanisms as explained in section 4 is discussed. We first explain the base model, after which we will cover each mechanism separately.

5.1 Applied Dataset

The simulations use the most recent uniform economic scenario set of De Nederlandsche Bank (De Nederlandsche Bank, 2026a, 2026-Q2 release) , This is the same data set Dutch pension funds must use for the statutory feasibility test and for communicating expected outcomes to participants, which makes it a neutral, reproducible basis for our analysis. It contains 20,000 scenarios over 100 years at annual frequency, providing an equity return and Dutch price inflation per scenario and year. Rather than storing interest-rate curves directly, the set provides three state variables per scenario and fixed ϕ and Ψ matrices, from which the nominal and real curves are reconstructed. We validated this against the set's reference curves. As the analysis concerns a Dutch pension, only the Dutch inflation rates and reference curves are used.

5.2 Base Model

The mechanisms are evaluated in a individual account model of the SPR, simulated for one representative participant across all 20,000 scenarios from the DNB dataset. The base model contains no damping. There is full exposure to the equity return and full pass-through to the benefit. Formulas are derived from textual description of De Nederlandsche Bank (De Nederlandsche Bank, 2024).
A participant enters at age 25, accumulates a flat annual contribution until age 68, and then draws benefits until a fixed age of 95 (illustrative age of death). At retirement the capital K is converted to a benefit via a flat annuity factor on a constant projection return r^{proj}. We set the projection return to a constant $r^{\text{proj}} = 3\%$ and normalize the annual contribution to one, so that all benefits are expressed as multiples of the annual contribution. Let H denote the remaining life horizon in years. The initial annual pension benefit is then given by:

$$b_0 = \frac{K}{\sum_{j=1}^{H}(1 + r^{\text{proj}})^{-j}}$$

After retirement, benefits are adjusted annually to reflect realized investment returns relative to the projection return. The benefit in year $k = 1, 2, \ldots, H$ of retirement is given by:

$$b_k = b_{k-1} \cdot \frac{1 + r_k^{\text{alloc}}}{1 + r^{\text{proj}}}$$

Benefits are deflated by realized Dutch inflation to obtain real outcomes.

5.3 Model per Scenario

We will model the effects of four damping scenarios: the spreading mechanism, the solidarity return, the protection return and the age-dependent allocation of excess return. The projection return

and the collective payout phase are excluded from the combined model and the decomposition, as they are mainly features that support stability but do not directly absorb or smooth investment shocks.

1. Spreading Mechanism

The spreading mechanism amortizes each annual return shock over a fixed horizon of N years rather than passing it through at once. In each payout year the shock is defined as the deviation of the allocated return from the projection return:

$$\text{Shock} = \frac{1 + r_k^{\text{alloc}}}{1 + r^{\text{proj}}} - 1$$

This shock is divided into N equal parts. Each of the parts is released in one of the following N years, so that the benefit adjustment in any year is the sum of the parts still active from the current and preceding shocks. We set $N = 5$ and release parts without interest. The spreading is applied only in the payout phase, where stability is the main objective.

2. Solidarity Reserve

The solidarity reserve is a collective buffer that directs a share of surpluses in favorable years and supplements benefits in adverse years. Since we use a single-cohort model, this is represented in reduced form as a participant-level buffer: in each payout year a surplus relative to the previous benefit is partly directed into the reserve, up to a cap. Using this method, the volatility reduction effect is isolated. We assume a direction rate of 25%, a cap of twice the annual benefit, and an initially empty reserve.

3. Protection Return

The protection return replaces part of the participant's equity exposure with an interest-rate hedge, governed by an age-dependent hedge ratio $h(a)$. The allocated return becomes

$$r^{\text{alloc}} = h(a)\, r^{\text{prot}} + \big(1 - h(a)\big)\, r^{\text{eq}}$$

where r^{prot} is the one-year holding return on a matching nominal zero-coupon bond with price P and duration p, $r_t^{\text{prot}} = P_{t+1}^{(d-1)} / P_t^{(d)} - 1$, priced based on the reconstructed term structure.

4. Age-dependent Allocation of Excess Return (Lifecycle)

This mechanism implements the lifecycle principle by allocating the risky excess return in proportion to an age-dependent share $w(a)$, with the remainder earning the risk-free rate:

$$r_{\text{alloc}} = w(a)\, r_{\text{eq}} + \big(1 - w(a)\big)\, r$$

The exposure is held at full risk until age 35 and then declines linearly to the minimum of 0.2, so that younger participants bear most of the investment risk while older participants bear less risk. Exposure is capped at 100% so there is no leveraging allowed.

5.4 Experimental Design

The model is evaluated both by (1) switching the isolating the effects of the four damping mechanism and (2) activating all four mechanisms at once, and comparing the resulting pension outcomes against the base model. When the four mechanisms are activated together, a fixed within-year order is applied: the allocated return is determined first, the raw benefit update next, then spreading, and finally the reserve, which acts on the shock remaining after spreading. This ordering is a modeling choice that affects the combined outcome.

5.4.1 Configurations

Three types of configuration are used. The baseline has all damping mechanisms turned off. The standalone configurations activate one mechanism at a time, isolating each mechanism's individual effect. The full factorial set runs all $2^4 = 16$ on/off combinations, which underlies the Shapley decomposition and includes both the baseline and the fully damped model as special cases. Each configuration is evaluated across all 20,000 scenarios.

5.4.2 Cohort grid

The analysis is carried out for a grid of cohorts currently aged 25, 35, 45, 55, and 67, each experiencing the same scenarios. Older cohorts are given a starting capital equal to the contributions they would have paid since age 25, grown at a fixed (deterministic) return. Only the years from the present onward are simulated stochastically, so the comparison reflects each cohort's remaining uncertainty rather than its past. All configurations are evaluated across the entire cohort grid. Hence, results for the representative cohort at age 45 are highlighted where a single-age view suffices.

5.4.3 Risk metrics

Outcomes are assessed on two risk metrics that answer different questions. The cross-scenario level dispersion, the coefficient of variation of the real benefit across scenarios, captures the uncertainty in the benefit level and is driven mainly by the accumulation phase. The year-on-year volatility, the fluctuation of the benefit during payout, captures stability in retirement and is structurally similar across cohorts. Both are reported, as the relative importance of the mechanisms differs sharply between them. The mean and standard deviation of the real benefit across the 20,000 scenarios are reported alongside as descriptives, where the mean is the average annual payout expressed as a multiple of the annual contribution.

5.4.4 Decomposition and Interaction

Because the mechanisms interact, their combined effect is not the sum of their individual effects, so the standalone comparison is incomplete. The total reduction in each metric is therefore decomposed using the Shapley value, treating each mechanism as a player whose value function is the reduction achieved by a coalition of active mechanisms, $v(S) = \rho_{\varnothing} - \rho_S$. Each mechanism's Shapley value is its average marginal contribution over all 16 coalitions, and by construction these values sum exactly to the full-model reduction (efficiency). The decomposition is computed for

both metrics: per cohort for level dispersion, and for the representative cohort at age 45 for year-on-year volatility, which does not vary across cohorts). Comparing the standalone effects with the Shapley values quantifies how much of each mechanism's effect is due to interaction rather than to the mechanism acting alone.

6 Results

This section evaluates the results from the four SPR damping mechanisms using 20,000 DNB scenarios. We distinguish two risk metrics in our results.

The first is the level dispersion, which is measured by the coefficient of variation (CoV). The level dispersion is calculated as the standard deviation divided by the mean pension outcome across the 20,000 scenarios. It describes the uncertainty in the pension level across the 20,000 scenarios and reflects how much the pension outcome varies to due long-term risks in the contribution phase.

The second is the year-on-year (YoY) volatility during the payout phase. It measure the volatility in pension benefits between two consecutive years during retirement and describes the short-term instability in the payout phase.

6.1 Baseline and Individual Mechanisms

Table 1 reports the standalone effects of each damping mechanism for a single cohort (age 45). In the baseline situation, no damping mechanism is activated, while mechanisms are activated in isolation in addition to the baseline setting in subsequent configurations. The outcomes per configuration are reported in terms of mean pension level, standard deviation, coefficient of variation (CoV), and year-on-year volatility.

Table 1: Baseline outcome and standalone effect of each mechanism, for the age 45 cohort. ΔCoV is the reduction in the CoV relative to the baseline.

Configuration	Mean	Std	CoV	ΔCoV	YoY Volatililty
Baseline (no damping)	8.2928	9.3117	1.0594	—	0.150
Protection return	2.9729	1.4570	0.4104	0.6490	0.056
Lifecycle	4.0731	2.4207	0.5961	0.4633	0.063
Spreading	8.5565	9.1418	0.9705	0.0889	0.058
Reserve	7.5637	7.7092	0.9803	0.0791	0.116

The baseline model shows substantial uncertainty with a CoV of 1.06. The dispersion of outcomes is as large as the expected pension level itself, indicating substantial uncertainty from full equity exposure during the contribution phase."

Activating the isolated protection return yields the strongest reduction in level dispersion, lowering the CoV from 1.06 to 0.41. This reflects the protection against interest-rate risk with an effective hedge, reducing long-run outcome uncertainty. The lifecycle mechanism also drastically reduces dispersion (CoV = 0.60), which is consistent with declining risk exposure at higher ages. The spreading mechanism and solidarity reserve have only minimal effects in reducing level dispersion, with CoV values of 0.97 and 0.98 respectively. This indicates limited impact on long-horizon outcome uncertainty when applied in isolation.

The reduction in risk goes with a change in expected pension levels. The protection return, for example, lowers the mean pension from 8.29 to 2.97. This reflects the trade-off between risk

reduction and pension outcomes. The lifecycle mechanism and the solidarity reserve also reduce the mean, but to a lesser extent.

If we study the short-term stability using the year-on-year volatility, we can see that all four mechanisms deliver effective smoothing effects, with the best strongest effects caused by the protection return, lifecycle mechanism and spreading mechanism. This shows that mechanisms which minimally reduce long-terms level dispersion can still be very effective in minimizing short-term stability.

6.2 Decomposition and Interaction

Table 2 reports the Shapley decomposition of the total damping effect, seperated for the contribution of each mechanism per age cohort. The reduction on the level dispersion (CoV) per mechanism is shown for each cohort. For the age 45 cohort, the reduction in year-on-year volatility per mechanism is shown. Because the mechanisms have non-linear interaction, the Shapley approach attributes the marginal contribution of each mechanism in a way that is order-independent and additive.

Table 2: Shapley decomposition of the volatility reduction, as percentage shares. Left: reduction in level dispersion per cohort; right: reduction in year-on-year volatility, shown for age 45 cohort.

	Level dispersion (CoV)					YoY volatility
Mechanism	25	35	45	55	67	45
Protection return	53.7%	55.1%	57.2%	62.0%	60.0%	27.5%
Lifecycle	37.2%	36.8%	34.2%	29.3%	26.1%	24.6%
Spreading	5.1%	4.5%	4.5%	4.8%	7.5%	37.9%
Reserve	4.0%	3.6%	4.0%	3.9%	6.3%	10.1%

For level dispersion, the results show a clear dominance of the protection return and lifecycle mechanisms in reducing long-term outcome uncertainty. Across all cohorts, the protection return accounts for approximately 54–62% of the total reduction in CoV, while the lifecycle mechanism contributes another 26–37%. The spreading mechanism and solidarity reserve play a relative minor role in reducing level dispersion, each contributing less than 8% in all cases.

The age gradient in the lifecycle contribution reflects its design. The impact reduces as age increases, reducing risk allocation for workers who are near retirement.

For year-on-year volatility for age cohort 45, the decomposition shows a different ranking. The spreading mechanism becomes the main driver in decreasing short-term volatility in the payout phase, accounting for $\tilde{4}0\%$ of the total reduction. The same trend can be observed for other age cohorts (not included in table). This is in line with the main objective of the spreading mechanism of providing stability during the payout phase. This reversal relative to the standalone results Table 1 reflects interaction: the protection return and lifecycle both reduce exposure to return shocks and are partial substitutes, while spreading smooths whatever shock remains and therefore retains its marginal value in every coalition.

6.3 Effect Across Cohorts

Table 3 presents the level dispersion (CoV) and year-on-year volatility across cohorts for the baseline and each mechanism applied in isolation. The results highlight how the mechanisms influence the two risk measures separately for each distinct age cohort.

Table 3: Level dispersion (CoV) for the baseline and for each mechanism activated on its own.

	Level dispersion (CoV)				
Configuration	25	35	45	55	67
Baseline	1.217	1.161	1.059	0.870	0.617
Protection return	0.624	0.564	0.410	0.216	0.088
Lifecycle	0.754	0.707	0.596	0.465	0.298
Spreading	1.123	1.080	0.971	0.782	0.520
Reserve	1.142	1.095	0.980	0.799	0.534

The first observable pattern is that level dispersion declines with age in the baseline case, from 1.22 at age 25 to 0.62 at age 67. This reflects the shortening investment horizon. As older cohorts are closer to retirement, they are exposed to less future shocks, reducing dispersion in final outcomes. This trend is observable for all mechanism, although at different levels.

The year-on-year volatility is essentially flat across cohorts and is therefore not included in Table 3. There are minor variations under the different mechanisms. This confirms that short-term payout instability is independent of age but is primarily driven by the return-to-benefit transmission mechanism in the payout phase.

Across mechanisms, the same ranking observed in Section 6.1 is preserved. Protection return consistently delivers the strongest reduction in level dispersion across all ages, followed by lifecycle, while spreading and the reserve have limited effects. The relative ordering is stable across cohorts, although the differences narrow at higher ages as baseline dispersion itself declines.

Finally, Table 4 illustrates the combined effect of all damping mechanisms simultaneously in the full model. The full model reduces level dispersion substantially across all cohorts compared to the baseline, with percentage reductions increasing from 56.5% at age 25 to 86.1% at age 67. This confirms that the combined mechanisms are highly effective and that their impact is particularly strong for older cohorts, consistent with the lifecycle-based reduction in residual risk exposure.

Table 4: Cohort comparison: level dispersion of the baseline versus the full damped model.

	25	35	45	55	67
Baseline CoV	1.217	1.161	1.059	0.870	0.617
Full Model CoV	0.529	0.470	0.330	0.172	0.086
Absolute reduction	0.688	0.691	0.729	0.697	0.532
% reduction	56.5	59.5	68.8	80.2	86.1

7 Discussion

7.1 Discussion

Together the four dampers strongly reduce the spread of pension outcomes across scenarios, and this effect grows with age. The reduction rises from 56.5% at age 25 to 86.1% at age 67 (Table 4). This is because older participants are further along the hedge and lifecycle paths and have little uncertainty left in their built-up capital, so the dampers protect participants closest to retirement the most.

Which mechanism does the work depends on the risk being measured, and the ranking changes between the two metrics (Table 2). The protection return and the lifecycle allocation are strongest for level dispersion, while spreading is strongest for year-on-year volatility in the payout phase. This follows from how the mechanisms work. Spreading and the reserve only act during payout, so they leave the accumulated capital untouched and barely reduce level dispersion. On year-on-year volatility they do better, since spreading spreads each shock over several years. Only the protection return scores well on both metrics, because it reduces risk both in the capital built up before retirement and in each year's update after retirement.

Reducing volatility comes at a price. De-risking replaces equity exposure with safer, lower-returning assets, so more damping means more stability but a lower expected benefit. The reserve and the lifecycle allocation also move risk between generations. Choosing a damping profile is not just a technical calibration. It also decides how risk is shared between generations.

7.2 Limitations

Three limitations affect how far these conclusions generalise. First, the model treats the reserve as a single person's own buffer, not as a pool shared across generations. A real reserve also moves money between age groups, which this model cannot show, so its effect here is a lower bound. Second, the hedge ratio $h(a)$ and the lifecycle exposure $w(a)$ are illustrative calibrations, not a real fund's policy. A different shape would likely keep the same ranking of mechanisms, but would shift the exact Shapley percentages. Third, all results rest on one set of 20,000 scenarios from Q2 2026, treated as if it covers every possible future. A different scenario set could give different numbers.

Five more simplifications affect the level of the results but not the ranking. The 100% equity baseline overstates both the risk and the return sacrifice relative to any realistic fund policy. Mortality is left out, and everyone is assumed to survive exactly to age 95, so the results do not capture the risk of dying early or living longer than expected. Contributions stay flat in nominal terms, so their real value falls over a career as inflation erodes them. The interest hedge uses a single bond matched to one point on the yield curve. It misses the finer shape of the curve, since it cannot capture short and long rates moving differently, while a full match across all maturities would. The other settings, such as the spreading horizon, the reserve's rules, and the fixed order of the mechanisms, are also illustrative rather than a real fund's policy. Still, the ranking of mechanisms follows from the basic logic of de-risking, not from these specific numbers.

8 Conclusion & Recommendations

8.1 Conclusion

The research question asked which volatility-reducing mechanisms are built into the SPR under the Wtp, and how large their individual and combined contributions to volatility reduction are. We modelled four mechanisms as volatility dampers: spreading, the solidarity reserve, the protection return, and the age-dependent allocation of excess return. The projection return and the collective payout phase were treated as level-shifting tools instead, so they shape the trajectory of the benefit rather than absorb shocks.

Together, the four dampers reduce the spread of pension outcomes by 56% to 86%, depending on cohort age, though which mechanism matters most depends on the metric. This damping is real and sizeable, but choosing it is a management-control decision. Lower volatility comes with a lower expected benefit and shifts risk between generations. So a fund should choose its mix based on which risk matters most to its own participants.

8.2 Recommendations

Funds should match the damping mix to what their population needs most. Near-retirees benefit most from spreading and the hedge, since these smooth the payout. Accumulating cohorts benefit most from the hedge and the lifecycle glide path, since these reduce level uncertainty. Because stability comes at the cost of the expected benefit, this trade-off should be communicated to participants clearly.

The reserve's funding and withdrawal rules should account for how they shift risk between generations, not just whether they reduce volatility for one person. This would require implementing a model with several age cohorts sharing one reserve at once, tracking contributions from one generation against withdrawals by another. Building this model is a natural next step for future research.

A Simulation Code

This appendix lists the full Python implementation of the simulation model described in the Methodology. The code was developed in a Jupyter notebook and is presented per step; each listing corresponds to one notebook cell. All simulations use the DNB scenario set described in Section 5.1.

A.1 Environment Setup

```
pip install python-calamine numpy pandas scipy matplotlib
```

A.2 Step 1: Loading the DNB Scenario Set and Reconstructing the Term Structure

```
Step 1 -- Load DNB CP2022-P scenario set + reconstruct the term structure.
First run reads the xlsx and writes a cache; later runs load from the cache.
import os
import numpy as np
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')

XLSX  =
    ↪ /content/drive/MyDrive/management_control_assignment_a/cp2022-p-scenarioset-20k-2026c
CACHE =
    ↪ /content/drive/MyDrive/management_control_assignment_a/scenario_cache.npz

SHEETS = [4_Aandelenrendement, 6_Prijsinflatie_NL,
          1_Toestandsvariabele_1, 2_Toestandsvariabele_2,
    ↪ 3_Toestandsvariabele_3,
          7_Renteparameter_phi_N, 8_Renteparameter_Psi_N,
          11_Renteparameter_phi_R_NL, 10_Renteparameter_Psi_R]
KEYS = [equity, infl_nl, s1, s2, s3, phi_N, Psi_N, phi_R_NL, Psi_R]

def read_sheet(name):
    arr = pd.read_excel(XLSX, sheet_name=name,
    ↪ header=None).to_numpy(dtype=float)
    arr = arr[~np.all(np.isnan(arr), axis=1)]
    arr = arr[:, ~np.all(np.isnan(arr), axis=0)]
    return arr

if os.path.exists(CACHE):
    z = np.load(CACHE)                                          # fast path
    arrays = {k: z[k] for k in KEYS}
else:
    arrays = {k: read_sheet(s) for k, s in zip(KEYS, SHEETS)}   # slow path,
    ↪ once
    np.savez_compressed(CACHE, **arrays)

# assign from `arrays` (no second Excel read)
equity, infl_nl = arrays[equity], arrays[infl_nl]
```

```
s1, s2, s3 = arrays[s1], arrays[s2], arrays[s3]
phi_N, Psi_N = arrays[phi_N], arrays[Psi_N]
phi_R_NL, Psi_R = arrays[phi_R_NL], arrays[Psi_R]
n_scen = equity.shape[0]

def curve(phi, Psi, t, maturities, scenarios):
    y(n,s,t) = exp( -1/n * ( phi[n,t] + sum_k S_k[s,t]*Psi[n,k] ) ) - 1
    idx = np.atleast_1d(maturities) - 1
    n = np.atleast_1d(maturities).astype(float)
    states = np.stack([s1[scenarios, t], s2[scenarios, t], s3[scenarios, t]],
    ↪ axis=1)
    return np.exp((-1.0 / n)[None, :] * (phi[idx, t][None, :] + states @
    ↪ Psi[idx, :].T)) - 1.0

def nominal_curve(t, maturities, scenarios): return curve(phi_N, Psi_N, t,
    ↪ maturities, scenarios)
def real_curve_nl(t, maturities, scenarios): return curve(phi_R_NL, Psi_R, t,
    ↪ maturities, scenarios)

print(equity.shape, infl_nl.shape)
print(np.round(nominal_curve(0, np.arange(1, 31), [0])[0] * 100, 2))
```

A.3 Step 2: Baseline Cohort Engine (No Damping)

```
Step 2 -- Cohort engine, baseline (no damping).
One cohort, vectorized over all scenarios. Assumes `equity` and `infl_nl` from
    ↪ step 1.

# --- contract settings ---
ENTRY_AGE      = 25
RETIREMENT_AGE = 68
END_AGE        = 95       # fixed horizon, no mortality yet (kept simple for
    ↪ the baseline)
CONTRIBUTION   = 1.0      # flat nominal contribution per accumulation year
    ↪ (scale is arbitrary)
PROJECTION     = 0.03     # flat projection return used to convert capital ->
    ↪ benefit

n_scen    = equity.shape[0]
acc_years = RETIREMENT_AGE - ENTRY_AGE    # accumulation years (43)
pay_years = END_AGE - RETIREMENT_AGE      # payout years (27)

def run_baseline(scenarios=None):
    Baseline SPR: 100% risky exposure, full pass-through, no damping.
    Returns (nominal_benefit, real_benefit), each shape (n_scen, pay_years).
    s = np.arange(n_scen) if scenarios is None else np.atleast_1d(scenarios)
    N = len(s)

    # --- accumulation: capital grows with the (undamped) equity return ---
    # Damping hooks here later: hedge return + age-dependent excess-return
    ↪ share.
    capital = np.zeros(N)
```

```
    for k in range(acc_years):                          # data year index t = k (0..42)
        capital = (capital + CONTRIBUTION) * (1.0 + equity[s, k])

    # --- conversion at retirement: flat-rate annuity factor ---
    # Damping hook later: projection return mechanism varies PROJECTION.
    horizon = np.arange(1, pay_years + 1)
    annuity_factor = np.sum((1.0 + PROJECTION) ** (-horizon))

    benefit = np.empty((N, pay_years))
    benefit[:, 0] = capital / annuity_factor

    # --- payout: variable benefit, full pass-through ---
    # Damping hooks here later: spreading smooths this update; reserve tops it up.
    for k in range(1, pay_years):
        t = acc_years + k - 1                           # data year index (43..68), contiguous
        benefit[:, k] = benefit[:, k - 1] * (1.0 + equity[s, t]) / (1.0 + PROJECTION)

    # --- deflate to real terms (NL inflation) ---
    price_level = np.cumprod(1.0 + infl_nl[s], axis=1)      # price level at each year-end
    pay_idx = np.clip(acc_years + np.arange(pay_years) - 1, 0, equity.shape[1] - 1)
    real_benefit = benefit / price_level[:, pay_idx]

    return benefit, real_benefit

# ---- quick check ----
nominal, real = run_baseline()
print(benefit shape:, real.shape)                                   # (20000, 27)
print(median real benefit, first payout year:, np.median(real[:, 0]).round(3))
print(5th / 50th / 95th pct, first year:,
      np.percentile(real[:, 0], [5, 50, 95]).round(3))
print(median year-on-year benefit volatility:,
      np.median(np.std(np.diff(real, axis=1) / real[:, :-1], axis=1)).round(3))
```

A.4 Step 3.1: Spreading Mechanism

```
Step 3 -- Mechanism #1: spreading mechanism (uitsmeren van schokken).
Spreads each year's return shock over N years instead of passing it through at once.
Sits on top of the baseline; assumes `equity`, `infl_nl` and the step-2 settings.

SPREAD_YEARS = 5      # N: number of years over which each shock is spread (lever; try 1/5/10)

def run_with_spreading(spread_years=SPREAD_YEARS, scenarios=None):
    Baseline payout, but each return shock is amortised over `spread_years`.
```

```
    Returns (nominal_benefit, real_benefit), each shape (n_scen, pay_years).

    Mechanics in the payout phase:
      raw shock in year k  : g_k = allocated_return / projection - 1
      that shock is released as g_k / N in each of the next N years
      benefit_k = benefit_at_retirement * product over active shock-slices

    s = np.arange(n_scen) if scenarios is None else np.atleast_1d(scenarios)
    N = len(s)
    M = max(int(spread_years), 1)

    # --- accumulation: identical to baseline (spreading only acts in payout) ---
    capital = np.zeros(N)
    for t in range(acc_years):
        capital = (capital + CONTRIBUTION) * (1.0 + equity[s, t])

    horizon = np.arange(1, pay_years + 1)
    annuity_factor = np.sum((1.0 + PROJECTION) ** (-horizon))
    first_benefit = capital / annuity_factor

    # --- payout: spread each yearly shock over M years ---
    benefit = np.empty((N, pay_years))
    benefit[:, 0] = first_benefit
    pending = np.zeros((N, M))            # ring buffer of shock-slices still to be released

    for k in range(1, pay_years):
        t = acc_years + k - 1
        raw_shock = (1.0 + equity[s, t]) / (1.0 + PROJECTION) - 1.0   # this year's shock
        pending[:, k % M] = raw_shock / M                              # split into M slices
        applied = pending.sum(axis=1)                                  # slices active this year
        benefit[:, k] = benefit[:, k - 1] * (1.0 + applied)

    price_level = np.cumprod(1.0 + infl_nl[s], axis=1)
    pay_data_years = acc_years + np.arange(pay_years)
    real_benefit = benefit / price_level[:, pay_data_years - 1]
    return benefit, real_benefit

# ---- quick check: spreading should lower year-on-year volatility vs baseline ----
def yoy_vol(real):
    return np.median(np.std(np.diff(real, axis=1) / real[:, :-1], axis=1))

_, real_base   = run_baseline()
_, real_spread = run_with_spreading(5)
print(yoy benefit volatility  baseline      :, round(yoy_vol(real_base), 3))
print(yoy benefit volatility  + spreading N=5:, round(yoy_vol(real_spread), 3))
```

A.5 Step 3.2: Solidarity Reserve (Single-Cohort Buffer)

```
Step 3 -- Mechanism #2: solidarity reserve (single-cohort buffer version).
A buffer that skims surplus in good years and tops up the benefit in bad years.
Captures the stabilising function only; intergenerational redistribution needs the
collective layer. Sits on top of the baseline.

RESERVE_FILL_RATE = 0.25    # fraction of each yearly surplus skimmed into the reserve
RESERVE_CAP       = 2.0     # reserve cap, expressed in units of one annual benefit

def run_with_reserve(fill_rate=RESERVE_FILL_RATE, cap=RESERVE_CAP, scenarios=None):
    Baseline payout, but a buffer smooths the benefit.
    Returns (nominal_benefit, real_benefit), each shape (n_scen, pay_years).

    Each payout year:
      raw      = benefit_prev * (1 + allocated) / (1 + projection)
      target   = benefit_prev                          (aim: avoid outright cuts)
      shortfall -> top up from reserve (if funds available)
      surplus  -> skim `fill_rate` into reserve, up to the cap

    s = np.arange(n_scen) if scenarios is None else np.atleast_1d(scenarios)
    N = len(s)

    # --- accumulation: identical to baseline ---
    capital = np.zeros(N)
    for t in range(acc_years):
        capital = (capital + CONTRIBUTION) * (1.0 + equity[s, t])

    horizon = np.arange(1, pay_years + 1)
    annuity_factor = np.sum((1.0 + PROJECTION) ** (-horizon))

    benefit = np.empty((N, pay_years))
    benefit[:, 0] = capital / annuity_factor
    reserve = np.zeros(N)                                  # buffer, starts empty

    for k in range(1, pay_years):
        t = acc_years + k - 1
        raw = benefit[:, k - 1] * (1.0 + equity[s, t]) / (1.0 + PROJECTION)
        target = benefit[:, k - 1]                         # hold the benefit flat where possible

        # bad year: draw from reserve to cushion the fall
        shortfall = np.maximum(target - raw, 0.0)
        topup     = np.minimum(shortfall, reserve)
        b = raw + topup
        reserve -= topup

        # good year: skim part of the surplus into the reserve, up to the cap
        surplus = np.maximum(raw - target, 0.0)
```

```
        room    = np.maximum(cap * benefit[:, k - 1] - reserve, 0.0)
        contrib = np.minimum(fill_rate * surplus, room)
        b -= contrib
        reserve += contrib

        benefit[:, k] = b

    price_level = np.cumprod(1.0 + infl_nl[s], axis=1)
    pay_data_years = acc_years + np.arange(pay_years)
    real_benefit = benefit / price_level[:, pay_data_years - 1]
    return benefit, real_benefit

# ---- quick check: reserve should lower volatility, at a small cost to the median ----
def yoy_vol(real):
    return np.median(np.std(np.diff(real, axis=1) / real[:, :-1], axis=1))

_, real_base    = run_baseline()
_, real_reserve = run_with_reserve()
print(yoy volatility  baseline :, round(yoy_vol(real_base), 3))
print(yoy volatility  + reserve:, round(yoy_vol(real_reserve), 3))
print(median real benefit  baseline / reserve:,
      round(np.median(real_base[:, 0]), 3), /,
      round(np.median(real_reserve[:, 0]), 3))
```

A.6 Step 3.3: Protection Return (Interest-Rate Hedge)

```
Step 3 -- Mechanism #3: protection return (interest-rate hedge).
Splits the allocated return into a hedged part and an equity part via an
age-dependent hedge ratio h(age). Uses the reconstructed nominal curve from step 1.
Sits on top of the baseline.

def hedge_ratio(age):
    Fraction of the return that comes from the interest hedge instead of equity.
    Linear ramp from 0 at age 45 to 1.0 at retirement; flat at 1.0 thereafter.
    return float(np.clip((age - 45) / (RETIREMENT_AGE - 45), 0.0, 1.0))

def liability_duration(age):
    Crude duration of the remaining benefit stream: midpoint of years left.
    years_left = max(END_AGE - age, 1)
    return max(years_left / 2.0, 1.0)

def discount(t, maturity, scenarios):
    Nominal zero-coupon discount factor P_t(maturity) = (1 + y)^(-maturity).
    if maturity <= 0:
        return np.ones(len(np.atleast_1d(scenarios)))
    y = nominal_curve(t, [maturity], scenarios)[:, 0]
    return (1.0 + y) ** (-maturity)

def protection_return(t, scenarios, duration):
```

```
    One-year holding return on a matching zero-coupon bond, earned over data year t.
    Buy a `duration`-year bond at t; one year later it is a (duration-1)-year bond.
    r_prot = P_{t+1}(d-1) / P_t(d) - 1.  Returns shape (n_scen,).
    d = int(round(duration))
    return discount(t + 1, d - 1, scenarios) / discount(t, d, scenarios) - 1.0

def run_with_hedge(scenarios=None):
    Baseline, but the allocated return is r = h*r_prot + (1-h)*r_equity.
    Returns (nominal_benefit, real_benefit), each shape (n_scen, pay_years).
    s = np.arange(n_scen) if scenarios is None else np.atleast_1d(scenarios)
    N = len(s)

    # --- accumulation with age-dependent hedging ---
    capital = np.zeros(N)
    for t in range(acc_years):
        age = ENTRY_AGE + t
        h = hedge_ratio(age)
        eq = equity[s, t]
        if h > 0:
            prot = protection_return(t, s, liability_duration(age))
            allocated = h * prot + (1.0 - h) * eq
        else:
            allocated = eq
        capital = (capital + CONTRIBUTION) * (1.0 + allocated)

    horizon = np.arange(1, pay_years + 1)
    annuity_factor = np.sum((1.0 + PROJECTION) ** (-horizon))

    # --- payout with hedging (retirees are fully hedged, h = 1) ---
    benefit = np.empty((N, pay_years))
    benefit[:, 0] = capital / annuity_factor
    for k in range(1, pay_years):
        t = acc_years + k - 1
        age = RETIREMENT_AGE + k - 1
        h = hedge_ratio(age)
        prot = protection_return(t, s, liability_duration(age))
        allocated = h * prot + (1.0 - h) * equity[s, t]
        benefit[:, k] = benefit[:, k - 1] * (1.0 + allocated) / (1.0 + PROJECTION)

    price_level = np.cumprod(1.0 + infl_nl[s], axis=1)
    pay_data_years = acc_years + np.arange(pay_years)
    real_benefit = benefit / price_level[:, pay_data_years - 1]
    return benefit, real_benefit

# ---- quick check: hedge should lower volatility vs baseline ----
def yoy_vol(real):
    return np.median(np.std(np.diff(real, axis=1) / real[:, :-1], axis=1))

_, real_base  = run_baseline()
_, real_hedge = run_with_hedge()
print(yoy volatility  baseline:, round(yoy_vol(real_base), 3))
```

```
print(yoy volatility  + hedge :, round(yoy_vol(real_hedge), 3))
```

A.7 Step 3.4: Age-Dependent Allocation of Excess Return (Lifecycle)

```
Step 3 -- Mechanism #4: age-dependent allocation of the excess return.
Younger participants get a larger share w(age) of the risky excess return,
older participants a smaller share (the remainder earns the risk-free return).
Sits on top of the baseline; the hedge (mechanism 3) is held neutral here.

def exposure(age):
    Share w(age) of the excess (equity) return allocated to the participant.
    Full exposure until age 35, then a linear glide path down to 0.2 at the
    ↪ end age.
    return float(np.clip(1.0 - 0.8 * (age - 35) / (END_AGE - 35), 0.2, 1.0))

def risk_free_1y(t, scenarios):
    One-year nominal risk-free rate at data year t, from the reconstructed
    ↪ curve.
    return nominal_curve(t, [1], scenarios)[:, 0]

def run_with_lifecycle(scenarios=None):
    Allocated return = w(age)*equity + (1 - w(age))*risk_free.
    Returns (nominal_benefit, real_benefit), each shape (n_scen, pay_years).
    s = np.arange(n_scen) if scenarios is None else np.atleast_1d(scenarios)
    N = len(s)

    # --- accumulation with age-dependent equity exposure ---
    capital = np.zeros(N)
    for t in range(acc_years):
        age = ENTRY_AGE + t
        w = exposure(age)
        rf = risk_free_1y(t, s)
        allocated = w * equity[s, t] + (1.0 - w) * rf
        capital = (capital + CONTRIBUTION) * (1.0 + allocated)

    horizon = np.arange(1, pay_years + 1)
    annuity_factor = np.sum((1.0 + PROJECTION) ** (-horizon))

    # --- payout: retirees keep a small, capped equity exposure (w -> 0.2) ---
    benefit = np.empty((N, pay_years))
    benefit[:, 0] = capital / annuity_factor
    for k in range(1, pay_years):
        t = acc_years + k - 1
        age = RETIREMENT_AGE + k - 1
        w = exposure(age)
        rf = risk_free_1y(t, s)
        allocated = w * equity[s, t] + (1.0 - w) * rf
        benefit[:, k] = benefit[:, k - 1] * (1.0 + allocated) / (1.0 +
    ↪ PROJECTION)

    price_level = np.cumprod(1.0 + infl_nl[s], axis=1)
    pay_data_years = acc_years + np.arange(pay_years)
    real_benefit = benefit / price_level[:, pay_data_years - 1]
```

```
    return benefit, real_benefit

# ---- quick check: lifecycle de-risking should lower volatility, especially for older ages ----
def yoy_vol(real):
    return np.median(np.std(np.diff(real, axis=1) / real[:, :-1], axis=1))

_, real_base      = run_baseline()
_, real_lifecycle = run_with_lifecycle()
print(yoy volatility  baseline   :, round(yoy_vol(real_base), 3))
print(yoy volatility  + lifecycle:, round(yoy_vol(real_lifecycle), 3))
print(median real benefit  baseline / lifecycle:,
      round(np.median(real_base[:, 0]), 3), /,
      round(np.median(real_lifecycle[:, 0]), 3))
```

A.8 Step 3.5: Projection Return (Level Lever, Excluded from Damping Set)

```
Step 3 -- Mechanism #5: projection return.
The projection return sets the annuity conversion factor and the yearly benefit
update. It is a level/trajectory lever, not a shock damper: a higher projection
return front-loads the benefit at the cost of a downward drift, and vice versa.
Sits on top of the baseline; `projection` is now an explicit argument.

def run_with_projection(projection, scenarios=None):
    Baseline mechanics, but the projection return is a chosen parameter.
    Returns (nominal_benefit, real_benefit), each shape (n_scen, pay_years).

    projection : annual projection return used both for the conversion annuity
                 factor and for the yearly benefit update benefit *= (1+r)/(1+proj).
                 Try e.g. the risk-free curve level (~0.025), 0.03, 0.04.

    s = np.arange(n_scen) if scenarios is None else np.atleast_1d(scenarios)
    N = len(s)

    # --- accumulation: identical to baseline ---
    capital = np.zeros(N)
    for t in range(acc_years):
        capital = (capital + CONTRIBUTION) * (1.0 + equity[s, t])

    # --- conversion: annuity factor depends on the projection return ---
    horizon = np.arange(1, pay_years + 1)
    annuity_factor = np.sum((1.0 + projection) ** (-horizon))

    # --- payout: benefit drifts up/down depending on realised vs projection ---
    benefit = np.empty((N, pay_years))
    benefit[:, 0] = capital / annuity_factor
    for k in range(1, pay_years):
        t = acc_years + k - 1
```

```
        benefit[:, k] = benefit[:, k - 1] * (1.0 + equity[s, t]) / (1.0 + projection)

    price_level = np.cumprod(1.0 + infl_nl[s], axis=1)
    pay_data_years = acc_years + np.arange(pay_years)
    real_benefit = benefit / price_level[:, pay_data_years - 1]
    return benefit, real_benefit

# ---- quick check: vary the projection return; see the level/drift trade-off ----
def yoy_vol(real):
    return np.median(np.std(np.diff(real, axis=1) / real[:, :-1], axis=1))

for proj in (0.025, 0.03, 0.04):
    _, real = run_with_projection(proj)
    start = np.median(real[:, 0])
    end   = np.median(real[:, -1])
    print(fprojection={proj:.3f}  median start={start:6.3f}  median end={end:6.3f}
          fdrift={end/start-1:+.1%}  yoy_vol={yoy_vol(real):.3f})
```

A.9 Step 3.6: Collective Payout Phase (Longevity Sharing, Excluded from Damping Set)

```
Step 3 -- Mechanism #6: collective payout phase (longevity sharing).
Introduces mortality in the payout phase and redistributes the capital of the
deceased across survivors as a biometric (mortality) return. This captures the
longevity-pooling part of the collective payout phase; cross-sectional return
pooling requires the multi-cohort collective layer. Sits on top of the baseline.

# Simple illustrative annual mortality rate by age in the payout phase.
# Replace with an AG prognosetafel for a realistic version.
def mortality_rate(age):
    Crude annual death probability q(age): roughly doubles every ~9 years.
    return float(np.clip(0.005 * 2 ** ((age - 65) / 9.0), 0.0, 1.0))

def run_with_longevity(scenarios=None):
    Baseline payout, but survivors receive the released capital of the deceased.
    Returns (nominal_benefit, real_benefit), each shape (n_scen, pay_years).

    Survival is deterministic at the cohort level: a fraction q(age) of the group
    dies each year, and their share is spread over the survivors, scaling up the
    per-survivor benefit by 1 / (1 - q). This is the longevity ('mortality') return.

    s = np.arange(n_scen) if scenarios is None else np.atleast_1d(scenarios)
    N = len(s)

```

```
    # --- accumulation: identical to baseline ---
    capital = np.zeros(N)
    for t in range(acc_years):
        capital = (capital + CONTRIBUTION) * (1.0 + equity[s, t])

    # annuity factor now accounts for survival probabilities (lower factor ->
    ↪ higher benefit)
    surv = 1.0
    survival = []
    for k in range(pay_years):
        survival.append(surv)
        surv *= (1.0 - mortality_rate(RETIREMENT_AGE + k))
    survival = np.array(survival)                              # prob. of being
    ↪ alive at payout year k
    horizon = np.arange(1, pay_years + 1)
    annuity_factor = np.sum(survival * (1.0 + PROJECTION) ** (-horizon))

    benefit = np.empty((N, pay_years))
    benefit[:, 0] = capital / annuity_factor
    for k in range(1, pay_years):
        t = acc_years + k - 1
        q = mortality_rate(RETIREMENT_AGE + k - 1)
        mortality_return = 1.0 / (1.0 - q)                     # released capital
    ↪ of the deceased
        allocated = (1.0 + equity[s, t]) * mortality_return
        benefit[:, k] = benefit[:, k - 1] * allocated / (1.0 + PROJECTION)

    price_level = np.cumprod(1.0 + infl_nl[s], axis=1)
    pay_data_years = acc_years + np.arange(pay_years)
    real_benefit = benefit / price_level[:, pay_data_years - 1]
    return benefit, real_benefit

# ---- quick check: longevity sharing lifts the benefit level; volatility ~
    ↪ unchanged ----
def yoy_vol(real):
    return np.median(np.std(np.diff(real, axis=1) / real[:, :-1], axis=1))

_, real_base = run_baseline()
_, real_long = run_with_longevity()
print(yoy volatility  baseline / longevity:,
      round(yoy_vol(real_base), 3), /, round(yoy_vol(real_long), 3))
print(median real benefit  baseline / longevity:,
      round(np.median(real_base[:, 0]), 3), /, round(np.median(real_long[:,
    ↪ 0]), 3))
```

A.10 Step 4: Combined Engine with Switchable Mechanisms

```
Step 4 -- Combined runner: all four real dampers as on/off flags in one
    ↪ engine.
With all flags off this reproduces run_baseline exactly. Mechanisms 5
    ↪ (projection)
and 6 (longevity) are kept out (level levers, handled as separate
    ↪ sensitivities).
```

```
4
5  # --- design profiles (illustrative levers) ---
6  def hedge_ratio(age):        return float(np.clip((age - 45) /
       ↪ (RETIREMENT_AGE - 45), 0.0, 1.0))
7  def exposure(age):           return float(np.clip(1.0 - 0.8 * (age - 35) /
       ↪ (END_AGE - 35), 0.2, 1.0))
8  def liability_duration(age): return max(max(END_AGE - age, 1) / 2.0, 1.0)
9
10 def discount(t, maturity, scenarios):
11     if maturity <= 0:
12         return np.ones(len(np.atleast_1d(scenarios)))
13     return (1.0 + nominal_curve(t, [maturity], scenarios)[:, 0]) **
       ↪ (-maturity)
14
15 def protection_return(t, scenarios, duration):
16     d = int(round(duration))
17     return discount(t + 1, d - 1, scenarios) / discount(t, d, scenarios) - 1.0
18
19 def risk_free_1y(t, scenarios):
20     return nominal_curve(t, [1], scenarios)[:, 0]
21
22 def run(hedge=False, lifecycle=False, spreading=False, reserve=False,
23         spread_years=5, fill_rate=0.25, cap=2.0, projection=PROJECTION,
24         entry_age=ENTRY_AGE, scenarios=None):
25     SPR cohort engine with switchable damping mechanisms.
26
27     Within-year order (locked in):
28       1. allocation : allocated = h*protection + (1-h)*[w*equity +
       ↪ (1-w)*risk_free]
29       2. raw update : raw_shock = (1+allocated)/(1+projection) - 1
30       3. spreading  : amortise raw_shock over `spread_years`
31       4. reserve    : top up shortfalls / skim surpluses on what remains
32
33     Flags off => h=0, w=1 => allocated=equity, no spreading, no reserve =>
       ↪ baseline.
34     Returns (nominal_benefit, real_benefit), each shape (n_scen, pay_years).
35
36     s = np.arange(n_scen) if scenarios is None else np.atleast_1d(scenarios)
37     N = len(s); M = max(int(spread_years), 1)
38     acc = RETIREMENT_AGE - entry_age
39
40     def allocate(t, age):
41         eq = equity[s, t]
42         w = exposure(age) if lifecycle else 1.0
43         h = hedge_ratio(age) if hedge else 0.0
44         risky = w * eq + (1.0 - w) * (risk_free_1y(t, s) if w < 1.0 else 0.0)
45         if h > 0.0:
46             return h * protection_return(t, s, liability_duration(age)) +
       ↪ (1.0 - h) * risky
47         return risky
48
49     # accumulation
50     capital = np.zeros(N)
51     for t in range(acc):
```

```
        capital = (capital + CONTRIBUTION) * (1.0 + allocate(t, entry_age + t))

    # conversion
    horizon = np.arange(1, pay_years + 1)
    annuity_factor = np.sum((1.0 + projection) ** (-horizon))
    benefit = np.empty((N, pay_years)); benefit[:, 0] = capital / annuity_factor
    pending = np.zeros((N, M)); res = np.zeros(N)

    # payout
    for k in range(1, pay_years):
        t = acc + k - 1
        allocated = allocate(t, RETIREMENT_AGE + k - 1)
        raw_shock = (1.0 + allocated) / (1.0 + projection) - 1.0
        if spreading:
            pending[:, k % M] = raw_shock / M
            applied = pending.sum(axis=1)
        else:
            applied = raw_shock
        b = benefit[:, k - 1] * (1.0 + applied)
        if reserve:
            target = benefit[:, k - 1]
            topup = np.minimum(np.maximum(target - b, 0.0), res); b += topup; res -= topup
            room  = np.maximum(cap * benefit[:, k - 1] - res, 0.0)
            contrib = np.minimum(fill_rate * np.maximum(b - target, 0.0), room); b -= contrib; res += contrib
        benefit[:, k] = b

    price_level = np.cumprod(1.0 + infl_nl[s], axis=1)
    pay_data_years = acc + np.arange(pay_years)
    return benefit, benefit / price_level[:, pay_data_years - 1]

# ---- verify: all flags off == baseline, then single-mechanism volatilities ----
def yoy_vol(real): return np.median(np.std(np.diff(real, axis=1) / real[:, :-1], axis=1))
nb, rb = run_baseline()
print(all-off == baseline:, np.allclose(nb, run(False, False, False, False)[0]))
print(baseline yoy vol   :, round(yoy_vol(rb), 3))
for name, kw in [(hedge, dict(hedge=True)), (lifecycle, dict(lifecycle=True)),
                 (spreading, dict(spreading=True)), (reserve, dict(reserve=True))]:
    print(f  + {name:9s}:, round(yoy_vol(run(**kw)[1]), 3))
print(  all four  :, round(yoy_vol(run(True, True, True, True)[1]), 3))
```

A.11 Step 5: Cohort Grid

```
Step 5 -- Cohort grid: run the engine for participants of different ages today.
Each cohort is seeded with the capital it would already have accumulated; the
```

```
remaining years run forward stochastically. run_grid(25) reproduces the baseline.

GRID = [25, 35, 45, 55, 67]

def accumulated_to(age):
    Deterministic starting capital: contributions from ENTRY_AGE to `age`,
    grown at the projection return. No past shocks -> clean forward comparison.
    cap = 0.0
    for _ in range(age - ENTRY_AGE):
        cap = (cap + CONTRIBUTION) * (1.0 + PROJECTION)
    return cap

def run_grid(start_age, hedge=False, lifecycle=False, spreading=False, reserve=False,
             spread_years=5, fill_rate=0.25, cap=2.0, projection=PROJECTION, scenarios=None):
    Engine for a cohort currently aged `start_age`, seeded with accumulated capital.
    Same within-year order as run(); returns (nominal_benefit, real_benefit).
    s = np.arange(n_scen) if scenarios is None else np.atleast_1d(scenarios)
    N = len(s); M = max(int(spread_years), 1)
    acc = max(RETIREMENT_AGE - start_age, 0)

    def allocate(t, age):
        eq = equity[s, t]
        w = exposure(age) if lifecycle else 1.0
        h = hedge_ratio(age) if hedge else 0.0
        risky = w * eq + (1.0 - w) * (risk_free_1y(t, s) if w < 1.0 else 0.0)
        if h > 0.0:
            return h * protection_return(t, s, liability_duration(age)) + (1.0 - h) * risky
        return risky

    capital = np.full(N, accumulated_to(start_age))
    for t in range(acc):
        capital = (capital + CONTRIBUTION) * (1.0 + allocate(t, start_age + t))

    horizon = np.arange(1, pay_years + 1)
    annuity_factor = np.sum((1.0 + projection) ** (-horizon))
    benefit = np.empty((N, pay_years)); benefit[:, 0] = capital / annuity_factor
    pending = np.zeros((N, M)); res = np.zeros(N)

    for k in range(1, pay_years):
        t = acc + k - 1
        allocated = allocate(t, RETIREMENT_AGE + k - 1)
        raw_shock = (1.0 + allocated) / (1.0 + projection) - 1.0
        if spreading:
            pending[:, k % M] = raw_shock / M; applied = pending.sum(axis=1)
        else:
            applied = raw_shock
```

```
        b = benefit[:, k - 1] * (1.0 + applied)
        if reserve:
            target = benefit[:, k - 1]
            topup = np.minimum(np.maximum(target - b, 0.0), res); b += topup; res -= topup
            room  = np.maximum(cap * benefit[:, k - 1] - res, 0.0)
            contrib = np.minimum(fill_rate * np.maximum(b - target, 0.0), room); b -= contrib; res += contrib
        benefit[:, k] = b

    price_level = np.cumprod(1.0 + infl_nl[s], axis=1)
    pay_data_years = acc + np.arange(pay_years)
    return benefit, benefit / price_level[:, pay_data_years - 1]

# two volatility notions: within-payout (same for all ages) vs level dispersion (age effect)
def yoy_vol(real):   return np.median(np.std(np.diff(real, axis=1) / real[:, :-1], axis=1))
def level_cov(real): return np.median(np.std(real, axis=0) / np.mean(real, axis=0))

print(f  {'age':>4} {'baseline CoV':>12} {'all four CoV':>12} {'reduction':>10})
for a in GRID:
    cb = level_cov(run_grid(a)[1])
    cd = level_cov(run_grid(a, hedge=True, lifecycle=True, spreading=True, reserve=True)[1])
    print(f  {a:>4} {cb:>12.3f} {cd:>12.3f} {1 - cd/cb:>9.1%})
```

A.12 Step 6: Shapley Decomposition

```
Step 6 -- Shapley decomposition of volatility reduction.
For each cohort, computes each mechanism's average marginal contribution to the
reduction in a chosen risk metric, over all 2^4 = 16 on/off coalitions.
Run it on BOTH metrics: the ranking flips, which is the central result.

from itertools import combinations
from math import factorial

MECHS = [spreading, reserve, hedge, lifecycle]

def metric_value(age, active, metric):
    Value of `metric` for a cohort with the `active` mechanisms switched on.
    flags = {m: (m in active) for m in MECHS}
    return metric(run_grid(age, **flags)[1])

def shapley_for_age(age, metric):
    Shapley value of each mechanism's contribution to metric REDUCTION vs baseline.
    Value function v(S) = metric(baseline) - metric(coalition S).
    base = metric_value(age, frozenset(), metric)
    v = {frozenset(S): base - metric_value(age, frozenset(S), metric)
```

```
         for r in range(len(MECHS) + 1) for S in combinations(MECHS, r)}
    n = len(MECHS); phi = {m: 0.0 for m in MECHS}
    for m in MECHS:
        for r in range(len(MECHS)):
            for S in combinations([x for x in MECHS if x != m], r):
                S = frozenset(S)
                w = factorial(len(S)) * factorial(n - len(S) - 1) /
    ↪ factorial(n)
                phi[m] += w * (v[S | {m}] - v[S])
    return phi, v[frozenset(MECHS)]       # (per-mechanism values, full-model
    ↪ reduction)

def shapley_table(metric):
    Shapley values and percentage shares per cohort for the given metric.
    rows = []
    for age in GRID:
        phi, total = shapley_for_age(age, metric)
        row = {Age: age, Total reduction: total}
        row.update({f{m}_value: phi[m] for m in MECHS})
        row.update({f{m}_share: 100 * phi[m] / total if total else 0.0 for m
    ↪ in MECHS})
        rows.append(row)
    return pd.DataFrame(rows)

# build the two decompositions
shapley_df       = shapley_table(level_cov)    # accumulation /
    ↪ level-uncertainty story
shapley_df_yoy   = shapley_table(yoy_vol)      # payout / year-on-year story

# efficiency check: Shapley values must sum to the full-model reduction
val_cols = [f{m}_value for m in MECHS]
assert np.allclose(shapley_df[val_cols].sum(axis=1), shapley_df[Total
    ↪ reduction])
assert np.allclose(shapley_df_yoy[val_cols].sum(axis=1), shapley_df_yoy[Total
    ↪ reduction])

# tables 5/6 from your module now work:
table5 = shapley_df.copy()
table6 = shapley_df[[Age, spreading_share, reserve_share, hedge_share,
    ↪ lifecycle_share]].copy()
table6.columns = [Age, Spreading %, Reserve %, Protection Return %, Lifecycle
    ↪ %]
print(table6.round(1).to_string(index=False))
```

A.13 Step 7: Generation of Result Tables

```
FULL = dict(spreading=True, reserve=True, hedge=True, lifecycle=True)    # the
    ↪ full damped model

# TABLE 1 -- Baseline outcomes (unchanged; baseline = no flags)
table1 = pd.DataFrame([{
    Age: age, Mean: np.mean(p), Std: np.std(p), CoV: level_cov(p),
    P5: np.percentile(p, 5), P95: np.percentile(p, 95)
```

```
} for age in GRID for p in [run_grid(age)[1]]]).round(4)
display(table1)

# TABLE 2 -- Individual (standalone) mechanism effects at age 45  [level-CoV metric]
base45 = level_cov(run_grid(45)[1])
mechanisms = {Spreading: dict(spreading=True), Reserve: dict(reserve=True),
              Protection return: dict(hedge=True), Lifecycle: dict(lifecycle=True)}
table2 = pd.DataFrame([{
    Mechanism: name, Mean: np.mean(p), CoV: level_cov(p),
    Delta CoV: base45 - level_cov(p), Tail P95: np.percentile(p, 95)
} for name, st in mechanisms.items() for p in [run_grid(45, **st)[1]]]
).sort_values(Delta CoV, ascending=False).round(4)
display(table2)

# TABLE 3 -- Sequential (cumulative) addition at age 45  [order-dependent: see Shapley]
steps = [(None, {}), (Spreading, dict(spreading=True)),
         (+ Reserve, dict(spreading=True, reserve=True)),
         (+ Hedge, dict(spreading=True, reserve=True, hedge=True)),
         (+ Lifecycle, FULL)]
table3 = pd.DataFrame([{
    Step: name, CoV: level_cov(p), Reduction: base45 - level_cov(p),
    % Reduction: 100 * (base45 - level_cov(p)) / base45
} for name, st in steps for p in [run_grid(45, **st)[1]]]).round(3)
display(table3)

# TABLE 4 -- Cohort comparison: baseline vs FULL damped model  (the fixed one)
table4 = pd.DataFrame([{
    Age: age, Baseline CoV: level_cov(run_grid(age)[1]),
    Full CoV: level_cov(run_grid(age, **FULL)[1]),
    Abs reduction: level_cov(run_grid(age)[1]) - level_cov(run_grid(age, **FULL)[1]),
    % reduction: 100 * (1 - level_cov(run_grid(age, **FULL)[1]) / level_cov(run_grid(age)[1]))
} for age in GRID]).round(3)
display(table4)

# TABLE 5/6 -- Shapley (from cell 11)
display(shapley_df.round(4))
table6 = shapley_df[[Age, spreading_share, reserve_share, hedge_share, lifecycle_share]].copy()
table6.columns = [Age, Spreading %, Reserve %, Protection Return %, Lifecycle %]
display(table6.round(1))

# Shapley shares on the year-on-year payout metric (the flipped ranking)
table7 = shapley_df_yoy[[Age, spreading_share, reserve_share,
                         hedge_share, lifecycle_share]].copy()
table7.columns = [Age, Spreading %, Reserve %, Protection Return %, Lifecycle %]
display(table7.round(1))
```

References

Balter, A., & Werker, B. J. M. (2021, November). *Equal fluctuations in pension payments implementable in new dutch pension scheme* (Netspar Design Paper No. 192). Netspar. https://www.netspar.nl/wp-content/uploads/P20211018_Netspar-Design-Paper-192-WEB.pdf

Bovenberg, A. L., & Nijman, T. E. (2009). Developments in pension reform: The case of Dutch stand-alone collective pension schemes. *International Tax and Public Finance*, *16*(4), 443–467. https://doi.org/10.1007/s10797-009-9108-1

De Nederlandsche Bank. (2022). Combinatie vaste daling en het toepassen van een spreidingsperiode bij een variabele pensioenuitkering [Q&A, De Nederlandsche Bank]. https://www.dnb.nl/voor-de-sector/open-boek-toezicht/sectoren/pensioenfondsen/prudentieel-toezicht/premieovereenkomst-premieregeling-en-variabele-uitkering/qa/combinatie-vaste-daling-en-het-toepassen-van-een-spreidingsperiode-bij-een-variabele-pensioenuitkering/

De Nederlandsche Bank. (2023a). Hoe geeft pensioenuitvoerder beheerste invulling aan keuze variabel projectierendement en gelijke aanpassingen in uitkeringen? [Open Boek Toezicht, Q&A, De Nederlandsche Bank]. https://www.dnb.nl/voor-de-sector/open-boek-toezicht/sectoren/pensioenfondsen/verzamelpagina-transitie-wet-toekomst-pensioenen/contract/hoe-geeft-pensioenuitvoerder-beheerste-invulling-aan-keuze-variabel-projectierendement-gelijke-aanpassingen-uitkeringen/

De Nederlandsche Bank. (2023b). Toedelingsregels in de solidaire premieregeling. https://www.dnb.nl/voor-de-sector/open-boek-toezicht/sectoren/pensioenfondsen/verzamelpagina-transitie-wet-toekomst-pensioenen/contract/toedelingsregels-in-de-solidaire-premieregeling/

De Nederlandsche Bank. (2023c, June). Solidariteitsreserve en risicodelingsreserve. https://www.dnb.nl/voor-de-sector/open-boek-toezicht/sectoren/pensioenfondsen/verzamelpagina-transitie-wet-toekomst-pensioenen/contract/solidariteitsreserve-en-risicodelingsreserve/

De Nederlandsche Bank. (2023d, June). Toedelingsregels in de solidaire premieregeling. https://www.dnb.nl/voor-de-sector/open-boek-toezicht/sectoren/pensioenfondsen/verzamelpagina-transitie-wet-toekomst-pensioenen/contract/toedelingsregels-in-de-solidaire-premieregeling/

De Nederlandsche Bank. (2024). Verzamelpagina transitie wet toekomst pensioenen. https://www.dnb.nl/voor-de-sector/open-boek-toezicht/sectoren/pensioenfondsen/verzamelpagina-transitie-wet-toekomst-pensioenen/

De Nederlandsche Bank. (2026a). Dnb publiceert definitieve scenariosets bij wet toekomst pensioenen (scenariosets 2026q2) [Open Boek Toezicht]. https://www.dnb.nl/voor-de-sector/open-boek-toezicht/sectoren/pensioenfondsen/dnb-publiceert-definitieve-scenariosets-bij-wet-toekomst-pensioenen/#scenariosets-2026q2

De Nederlandsche Bank. (2026b). Ons pensioenstelsel straks. https://www.dnb.nl/actuele-economische-vraagstukken/pensioen/ons-pensioenstelsel-straks/

Ministerie van Sociale Zaken en Werkgelegenheid. (2022). Wijziging van de Pensioenwet, de Wet inkomstenbelasting 2001 en enige andere wetten in verband met herziening van het pensioenstelsel: Memorie van toelichting. https://zoek.officielebekendmakingen.nl/kst-36067-3.pdf

van Bilsen, S., & Mehlkopf, R. J. (2026). *Solidariteitsreserve* (Netspar Industry Paper No. 2026-17). Netspar. https://www.netspar.nl/wp-content/uploads/Netspar-Industry-Paper-2026-17-WEB-1.pdf

van Bilsen, S., Mehlkopf, R. J., & van Stalborch, S. (2022). Intergenerational transfers in the new Dutch pension contract. *De Economist*, *170*(1), 37–67. https://doi.org/10.1007/s10645-022-09399-4

Werken aan ons Pensioen. (n.d.-a). Flexibele premieregeling. https://www.werkenaanonspensioen.nl/onderwerpen/c/contractkeuze-pf-en-wg-wn/flexibele-premieregeling

Werken aan ons Pensioen. (n.d.-b). Uitkeringsfase – solidaire premieregeling. https://www.werkenaanonspensioen.nl/onderwerpen/c/contractkeuze-pf-en-wg-wn/solidaire-premieregeling/uitkeringsfase

Werker, B. (2022). *Beschermings-, over- en projectierendementen* (Board Brief). Netspar. https://www.netspar.nl/kennisplein/beschermings-over-en-projectierendementen/

2022, https://wetten.overheid.nl/BWBR0020809/2022-07-07